



Московский
транспорт

Система обнаружения посторонних объектов для беспилотных поездов в тоннеле метро по данным 3D-лидара

Техническое задание

2026

Система обнаружения посторонних объектов для беспилотных поездов в тоннеле метро по данным 3D-лидара

1. Контекст и актуальность задачи

Московский метрополитен — одна из крупнейших транспортных систем мира. Каждый день миллионы пассажиров доверяют ему свои поездки, а в ближайшем будущем метро должно стать ещё технологичнее: мы работаем над созданием беспилотного поезда, способного самостоятельно двигаться по тоннелю и принимать решения без машиниста.

У беспилотного поезда есть одна фундаментальная задача: он должен видеть дорогу перед собой.

Представьте поезд, который движется по тоннелю. Впереди может быть человек, упавший предмет, строительное оборудование или любое другое препятствие. Беспилотная система должна обнаружить потенциальную опасность заранее — чтобы у неё оставалось достаточно времени на реакцию и безопасное торможение.

Одним из основных «органов зрения» поезда является 3D-лидар. Он формирует облако точек окружающего пространства и позволяет буквально построить трёхмерную картину тоннеля на сотни метров вперёд.

И здесь начинается самое интересное.

Классические алгоритмы компьютерного зрения отлично чувствуют себя там, где вокруг много разнообразных объектов: деревья, здания, автомобили, люди. Тоннель метро — совсем другая история. Это длинное, однородное пространство с рельсами, стенами, кабелями и однотипными конструкциями. Большую часть времени перед поездом вообще ничего нет.

Поэтому нам недостаточно просто «найти человека на картинке».

Нам нужно научить систему отвечать на гораздо более важный вопрос:

«Есть ли что-нибудь на пути беспилотного поезда, чего там быть не должно?»

В рамках кейса вам предстоит попробовать решить эту задачу с помощью лидарных данных и предложить свой подход к обнаружению препятствий.

Мы специально не ограничиваем участников одним классом алгоритмов. Это может быть классическая обработка облаков точек, машинное обучение, нейронные сети, анализ аномалий, геометрические методы или комбинация нескольких подходов.

Мы ждём не столько правильного применения известного алгоритма, сколько интересных идей, способных работать в необычной среде метро.

2. Описание задачи

Что нужно сделать?

Разработать программное решение, которое по данным 3D-лидара определяет наличие потенциальных препятствий перед движущимся поездом.

В распоряжении участников будут записи реальных лидарных данных, полученные во время движения поезда в тоннеле.

На вход вашему алгоритму поступает поток точек 3D-лидара.

На выходе система должна определять, есть ли впереди поезда посторонний объект, представляющий потенциальную опасность.

В простейшем варианте результат может выглядеть как:

- препятствие обнаружено / препятствие не обнаружено;
- расстояние до ближайшего препятствия.

Более продвинутые решения могут дополнительно определять положение, размеры, тип или степень уверенности в обнаружении объекта.

Главный вызов

Тоннель — очень специфичная среда для лидарных алгоритмов.

Большая часть предоставленных записей будет содержать пустой тоннель без препятствий. Поэтому подходы, которые требуют большого количества размеченных примеров каждого возможного объекта, могут столкнуться с проблемой нехватки обучающих данных.

С другой стороны, именно это открывает пространство для нестандартных решений.

Например, можно попробовать искать:

- отклонения от привычной геометрии тоннеля;
- изменения в пространстве между последовательными кадрами;
- аномальные кластеры точек;
- объекты, пересекающие габарит движения поезда;
- препятствия, которые появляются в динамике;
- комбинации геометрических и ML-подходов.

Это лишь примеры направления мысли — мы не задаём конкретный способ решения.

Возможно, именно ваша команда найдёт способ, который мы сами ещё не пробовали.

Ограничения и свобода выбора

Можно использовать:

- классические алгоритмы обработки 3D-облаков;
- машинное обучение;
- нейронные сети;
- методы обнаружения аномалий;
- алгоритмы кластеризации и сегментации;
- временной анализ последовательности лидарных кадров;
- собственные математические модели;
- комбинацию нескольких подходов.

Можно использовать любой язык программирования, совместимый с ROS 2 Humble.

Главное — чтобы решение действительно работало на предоставленных данных и могло быть воспроизведено на нашем стенде.

Важный момент

Тестовые данные не будут полностью известны участникам.

Мы предоставим набор лидарных bag-файлов для разработки и отладки. Финальная проверка будет проводиться на контрольных данных, которые команды заранее не получат.

Поэтому решение должно быть не просто подогнано под конкретные записи, а способно обобщаться на новые проезды и новые ситуации.

3. Программно-аппаратные требования:

3.1. Тестовый стенд

Проверка решений будет выполняться на следующем оборудовании:

- **CPU:** Intel Core i7-9700E, 8 ядер, 2.60 GHz
- **RAM:** 128 GiB
- **GPU:** NVIDIA GeForce RTX 4070 Ti SUPER
- **Диск:** 5,8 TB
- **OS:** Ubuntu 22.04.5 LTS

GPU может использоваться алгоритмом, если это необходимо для его работы.

3.2. Программные требования

Обязательная среда:

- Ubuntu 22.04
- ROS 2 Humble
- Docker

Участники могут использовать любой язык программирования и любые библиотеки, которые можно установить и запустить внутри данной среды.

3.3. ROS 2 и данные

Лидарные данные будут предоставляться в виде ROS 2 bag-файлов.

Решение должно уметь:

1. Запуститься в Docker-контейнере.
2. Подключиться к ROS 2.
3. Прочитать предоставленный bag-файл.
4. Обработать поток лидарных данных.
5. Вывести результат работы алгоритма.

Допускается использование дополнительных ROS 2-пакетов и библиотек при условии, что они устанавливаются автоматически в рамках сборки Docker-образа.

4. Требования к презентации/демонстрации

На демонстрации команда должна показать работу своего решения на предоставленном лидарном bag-файле.

Минимальный сценарий:

1. Запустить Docker-контейнер с решением.
2. Начать проигрывание ROS 2 bag-файла.
3. Показать поступающие лидарные данные.
4. Продемонстрировать обнаружение препятствия.
5. Показать результат работы алгоритма.

Для визуализации можно использовать RViz2 или другое подходящее средство визуализации.

Особенно приветствуется демонстрация работы системы в реальном времени — например, через удалённый рабочий стол.

Нам хочется увидеть не только красивый слайд с архитектурой, но и цепочку:

тоннель → облако точек → алгоритм → обнаружено препятствие → расстояние до него.

5. Требования к сопроводительной документации

К решению необходимо приложить README, содержащий:

- описание проекта;
- инструкцию по сборке Docker-образа;
- инструкцию по запуску;
- описание запуска обработки bag-файла;
- описание необходимых параметров и конфигурации.

Также необходимо предоставить:

- Архитектуру решения

Кратко показать, из каких компонентов состоит система и как между ними передаются данные.

Например:

ROS 2 bag → обработка облака → алгоритм обнаружения → результат детекции → визуализация

- Описание алгоритма

Важно рассказать:

- какую проблему решает выбранный подход;

- какие данные используются;
- как происходит обработка облака;
- каким образом принимается решение о наличии препятствия;
- какие параметры влияют на результат;
- какие ограничения есть у вашего метода.

Если в основе решения лежит математическая модель — будет интересно увидеть её описание.

- Эксперименты

Желательно привести результаты собственных экспериментов:

- на какой дистанции удалось обнаруживать препятствия;
- какая задержка обработки;
- сколько кадров в секунду обрабатывает система;
- какие были ложные срабатывания;
- какие ситуации оказались сложными;
- как менялось качество после улучшения алгоритма.

Не бойтесь показывать не только успехи. Хороший инженерный разбор неудачного подхода может быть не менее интересным, чем финальный результат.

- Видео

Необходимо предоставить короткое видео работы алгоритма.

6. Ресурсы:

Участникам будут предоставлены:

- записи реальных проездов поезда в тоннеле в формате ROS 2 bag;
- необходимые описания формата входных данных;
- информация, необходимая для запуска решения;
- описание тестового окружения.

Конкретный состав и объём предоставляемых данных будут опубликованы организаторами дополнительно.

7. Требования к сдаче решений

7.1. Промежуточная сдача

К промежуточной сдаче необходимо предоставить:

- Dockerfile и/или готовый Docker-образ со всем необходимым для сборки и запуска;
- работоспособный прототип алгоритма;
- краткое описание выбранного подхода;

- минимальную демонстрацию работы на предоставленных данных;
- первые результаты экспериментов.

На этом этапе от команды не требуется идеальное решение.

Нам важно увидеть, какую гипотезу вы выбрали и почему считаете, что она может сработать.

7.2. Финальная сдача

К финальной сдаче необходимо предоставить:

- Docker-контейнер со всем необходимым для запуска;
- исходный код решения;
- README с инструкцией по запуску;
- описание архитектуры;
- описание алгоритма;
- результаты экспериментов;
- видео работы;
- полноценную демонстрацию решения на контрольном bag-файле.

Финальная версия должна запускаться в предоставленном окружении без ручной установки зависимостей.

8. Критерии оценки

Главный критерий очень простой:

Насколько хорошо ваш алгоритм позволяет беспилотному поезду увидеть препятствие заранее?

8.1. Работоспособность решения — основной критерий

Оцениваем:

- способность обнаруживать препятствия;
- устойчивость работы на различных участках тоннеля;
- количество пропущенных препятствий;
- количество ложных срабатываний;
- способность работать на данных, которые не использовались при разработке.

8.2. Дальность обнаружения

Чем раньше поезд увидит препятствие — тем лучше.

Особое внимание будет уделяться максимальной дистанции, на которой алгоритм способен уверенно обнаружить объект.

Условно:

300 м → отлично

200 м → очень хорошо

100 м → хорошо

Но итоговая оценка будет зависеть не только от дальности, но и от качества обнаружения.

Важно найти баланс между дальностью, надежностью и количеством ложных тревог.

8.3. Скорость работы

Беспилотному поезду мало просто знать, что препятствие когда-то было обнаружено.

Информация должна появиться достаточно быстро, чтобы система могла на неё отреагировать.

Поэтому мы будем смотреть на:

- задержку обработки;
- частоту обработки лидарных кадров;
- использование CPU/GPU;
- стабильность работы в реальном времени.

Чем ближе решение к работе в реальном времени — тем лучше.

8.4. Обобщаемость

Финальная проверка будет проводиться на данных, которые участники не увидят заранее.

Поэтому мы будем оценивать не способность «выучить конкретный bag», а способность алгоритма работать на новых данных.

Особенно интересно увидеть решения, которые используют особенности самой среды тоннеля, а не только запоминают внешний вид отдельных объектов.

8.5. Техническая проработка

Будем смотреть на:

- архитектуру решения;
- качество и читаемость кода;
- устойчивость;
- воспроизводимость;
- наличие тестов;
- использование ресурсов;
- качество документации;
- понятность алгоритма.

Отдельный плюс — возможность быстро разобраться в решении и запустить его на новом наборе данных.

8.6. Простота запуска

Представьте, что ваша команда передаёт решение инженеру метро.

Инженер должен иметь возможность сделать примерно следующее:

`docker build` → `docker run` → `ros2 bag play` → увидеть результат

Чем меньше ручных действий и «магии» требуется для запуска, тем лучше.

8.7. Подход команды

Нам интересно не только что получилось, но и как команда к этому пришла:

- какие гипотезы рассматривались;
- какие эксперименты проводились;
- какие подходы не сработали;
- почему был выбран финальный вариант;
- какие компромиссы пришлось сделать.

8.8. Питч-сессия

И наконец — презентация. Она будет иметь меньший вес, чем реальные технические результаты.

При этом хороший питч поможет показать сильные стороны решения. Будет здорово, если за несколько минут команда сможет понятно описать:

проблема → идея → алгоритм → демонстрация → результаты → что получилось.

А затем показать главное: «Вот поезд едет по тоннелю. Вот лидар. А вот препятствие, которое наш алгоритм увидел за X метров».

Что мы особенно хотим увидеть

Мы специально оставляем пространство для экспериментов и нестандартных подходов. Не обязательно строить очередную нейронную сеть, которая классифицирует point cloud.

Возможно, лучший способ решить задачу — вообще не пытаться классифицировать объекты. Возможно, нужно научиться хорошо описывать нормальный тоннель, а затем искать всё, что в него не вписывается. Возможно, ключом станет анализ движения. Возможно — геометрия рельсов и габарита. Возможно — комбинация нескольких простых методов окажется лучше одной сложной модели.

Мы не знаем заранее, какой подход победит. И именно поэтому предлагаем вам попробовать найти его.

Ваша задача — дать беспилотному поезду возможность посмотреть вперёд и сказать: «Путь свободен» или «Впереди препятствие. Нужно тормозить».